CSC258 Distributed Systems
Instructor: Dr. Abeer Abdel Khaleq
Spring 2026
Tam Vo

Implementation Assignment #3: Coordination

1. Based on your solution to programming assignment 2 that you worked on earlier, update the solution by adding a file on the server. Simulate having multiple clients accessing the file by adding or reading from the file. Run your code and show the results.

2. Discuss the approach that your solution took for the processes to communicate with the server and access the file?

3. Simulate increasing the number of processes accessing the file. What issues would you run into?

4. Based on the different coordination algorithms discussed in class, which one do you think will be a good solution for this problem? Explain your reasoning and how it applies to your solution.

5. Reflection on testing, observations, challenges, etc.

6. Submit the new updated code, fully documented with comments and resources used. You also need to submit the PDF showing all the testing you did on your code to support your answer, along with references.

Introduction (#1-2)

The environment that I used was Java 15+ with the package manager being Apache Maven. Using Docker, I created a compose.yml file that detailed the dependency order of the project. Alongside this I created Dockerfile.client and Dockerfile.server to have a custom startup with the client and server images respectively. With this after using ***docker compose up --build server*** command and the ***docker compose up --build --scale client=10 client*****,** I was able to deploy this simple server and client server. The Client program will randomly modify or read from a file on the server. The clients will be taking this to the extreme where the poll to read and modify the files on the server. The clients will communicate through basic networking protocols.

Problems (#3-4)

We will additionally scale from 1, 10 to 100. The issues that came with increasing number of users was that files would be corrupted during the writing when reading. To mitigate this, I created a RetreantRWLock for the file on the server. The coordination-based algorithms

that we would require to prevent these types of corruptions would be distributed locking and consensus algorithms such as ZooKeeper.

The server successfully built in Java and hard-coded exposed port 8080 for the clients to connect.

```
○ tvo@tvos-MacBook-Pro-5 HW2 % docker compose up --build server

  [+] Building 1.9s (16/16) FINISHED
   => [internal] load local bake definitions                                    0.0s
   => => reading from stdin 559B                                                0.0s
   => [internal] load build definition from Dockerfile.server                   0.0s
   => => transferring dockerfile: 271B                                          0.0s
   => resolve image config for docker-image://docker.io/docker/dockerfile:1     0.9s
   => [auth] docker/dockerfile:pull token for registry-1.docker.io              0.0s
   => CACHED docker-image://docker.io/docker/dockerfile:1@sha256:b6afd42430b1   0.0s
   => [internal] load metadata for docker.io/library/maven:latest               0.6s
   => [auth] library/maven:pull token for registry-1.docker.io                  0.0s
   => [internal] load .dockerignore                                             0.0s
   => => transferring context: 2B                                               0.0s
   => [1/5] FROM docker.io/library/maven:latest@sha256:4f82a03a7d6679281952d6   0.0s
   => [internal] load build context                                             0.0s
   => => transferring context: 3.48kB                                           0.0s
   => CACHED [2/5] WORKDIR /src                                                 0.0s
   => CACHED [3/5] COPY pom.xml pom.xml                                         0.0s
   => CACHED [4/5] RUN mvn clean install                                        0.0s
   => [5/5] COPY src/main/java/HW2 ./src/main/java/HW2                          0.0s
   => exporting to image                                                        0.0s
   => => exporting layers                                                       0.0s
   => => writing image sha256:c41ad16535108a57359e5bc2fa04fe07eec64534cb72b87   0.0s
   => => naming to docker.io/library/hw2-server                                 0.0s
   => resolving provenance for metadata file                                    0.0s
  [+] up 3/3
   ✔ Image hw2-server       Built                                              1.9s
   ✔ Network hw2_default    Created                                            0.0s
   ✔ Container hw2-server-1 Created                                            0.0s
  Attaching to server-1
```

The client successfully built in Java to connect to the exposed port 8080.

```
○ tvo@tvos-MacBook-Pro-5 HW3 % docker compose up --build --scale client=10 client

  [+] Building 1.8s (19/19) FINISHED
   => [internal] load local bake definitions
   => => reading from stdin 1.03kB
   => [client internal] load build definition from Dockerfile.client
   => => transferring dockerfile: 259B
   => [server internal] load build definition from Dockerfile.server
   => => transferring dockerfile: 271B
   => [server] resolve image config for docker-image://docker.io/docker/dockerfile:1
   => [auth] docker/dockerfile:pull token for registry-1.docker.io
   => CACHED [server] docker-image://docker.io/docker/dockerfile:1@sha256:b6afd42430b15f2d2a4c5a02b919e98a525b785b1aaff16747d2f623364e39b6
   => [server internal] load metadata for docker.io/library/maven:latest
   => [auth] library/maven:pull token for registry-1.docker.io
   => [server internal] load .dockerignore
   => => transferring context: 2B
   => [client 1/5] FROM docker.io/library/maven:latest@sha256:303967de98a80bb27e9fa71a97b3f0db8a1b30f4e80e5067569d346eb37dbe58
   => [server internal] load build context
   => => transferring context: 4.23kB
   => CACHED [server 2/5] WORKDIR /src
   => CACHED [server 3/5] COPY pom.xml pom.xml
   => CACHED [server 4/5] RUN mvn clean install
   => [server 5/5] COPY src/main/java/HW2 ./src/main/java/HW2
   => [client] exporting to image
   => => exporting layers
   => => writing image sha256:bc0a5ea23157b3d078f69567d0f152e9ea6718b1415d300f079439bab9dfa997
   => => naming to docker.io/library/hw3-client
   => [server] exporting to image
   => => exporting layers
   => => writing image sha256:7f0a1842b69f47f4f120284be444a8c517ff3c43e084d6bd060fc2f4d64fbd78
   => => naming to docker.io/library/hw3-server
   => [client] resolving provenance for metadata file
   => [server] resolving provenance for metadata file
  [+] up 13/13
   ✔ Image hw3-server        Built
   ✔ Image hw3-client        Built
   ✔ Container hw3-server-1  Recreated
   ✔ Container hw3-client-7  Recreated
   ✔ Container hw3-client-9  Recreated
   ✔ Container hw3-client-5  Recreated
   ✔ Container hw3-client-1  Recreated
   ✔ Container hw3-client-10 Recreated
   ✔ Container hw3-client-4  Recreated
   ✔ Container hw3-client-2  Recreated
   ✔ Container hw3-client-6  Recreated
   ✔ Container hw3-client-8  Recreated
   ✔ Container hw3-client-3  Recreated
  Attaching to client-1, client-10, client-2, client-3, client-4, client-5, client-6, client-7, client-8, client-9
  client-10  | [INFO] Scanning for projects...
```

```
client-10  | Client #86 sent: WRITE Hello, Server! from Client #86
client-9   | Client #21 sent: READ
client-6   | Client #75 sent: WRITE Hello, Server! from Client #75
client-7   | Client #65 sent: READ
client-2   | Client #67 sent: WRITE Hello, Server! from Client #67
client-1   | Client #40 sent: WRITE Hello, Server! from Client #40
client-5   | Client #91 sent: READ
client-4   | Client #92 sent: READ
client-8   | Client #38 sent: WRITE Hello, Server! from Client #38
client-10  | Client #86 sent: WRITE Hello, Server! from Client #86
client-9   | Client #21 sent: READ
client-6   | Client #75 sent: READ
client-7   | Client #65 sent: WRITE Hello, Server! from Client #65
client-2   | Client #67 sent: WRITE Hello, Server! from Client #67
client-1   | Client #40 sent: READ
client-5   | Client #91 sent: WRITE Hello, Server! from Client #91
client-4   | Client #92 sent: READ
client-8   | Client #38 sent: WRITE Hello, Server! from Client #38
client-10  | Client #86 sent: READ
client-9   | Client #21 sent: READ
client-6   | Client #75 sent: WRITE Hello, Server! from Client #75
client-7   | Client #65 sent: WRITE Hello, Server! from Client #65
client-2   | Client #67 sent: READ
client-1   | Client #40 sent: WRITE Hello, Server! from Client #40
client-5   | Client #91 sent: READ
client-4   | Client #92 sent: WRITE Hello, Server! from Client #92
client-8   | Client #38 sent: READ
client-10  | Client #86 sent: READ
client-9   | Client #21 sent: READ
client-6   | Client #75 sent: READ
client-7   | Client #65 sent: READ
client-2   | Client #67 sent: READ
client-1   | Client #40 sent: WRITE Hello, Server! from Client #40
client-5   | Client #91 sent: READ
client-4   | Client #92 sent: READ
client-8   | Client #38 sent: WRITE Hello, Server! from Client #38
client-10  | Client #86 sent: WRITE Hello, Server! from Client #86
client-9   | Client #21 sent: WRITE Hello, Server! from Client #21
client-6   | Client #75 sent: WRITE Hello, Server! from Client #75
client-7   | Client #65 sent: WRITE Hello, Server! from Client #65
client-2   | Client #67 sent: READ
```

Reflection of Testing, Observations and Challenges

Testing was done before the deployment of the Server and Client. I used Apache Maven to test the Client and Server Application by mocking the expected output with the modified input or function call using the Java Library Mocktio. Using **mvn clean test,** you can run the

tests created for the Client and server. Mocking the client needed to be updated since there should be a flag to notate sleeping messages or not from the previous assignment.

From my observations, the docker containers seems to be installing the same packages to the client images. This is interesting since I each image should be the same, but why is it that they are having duplicated installs without copying from another image to another.

One of the biggest challenges of this project was learning the coordination of the docker server container. When running multiple clients it would corrupt the file, but it was fast, but when implementing the lock, the read and writes were slower.

Testing

```
● tvo@tvos-MacBook-Pro-5 HW3 % mvn test
  [INFO] Scanning for projects...
  [INFO]
  [INFO] --------------< com.tamkhaivo:distributed-systems-hw1 >---------------
  [INFO] Building distributed-systems-hw1 1.0-SNAPSHOT
  [INFO]   from pom.xml
  [INFO] --------------------------------[ jar ]---------------------------------
  [INFO]
  [INFO] --- dependency:3.6.1:properties (default) @ distributed-systems-hw1 ---
  [INFO]
  [INFO] --- resources:3.3.1:resources (default-resources) @ distributed-systems-hw1 ---
  [INFO] skip non existing resourceDirectory /Users/tvo/Documents/portfolio/Distributed-Systems/HW3/src/main/resources
  [INFO]
  [INFO] --- compiler:3.13.0:compile (default-compile) @ distributed-systems-hw1 ---
  [INFO] Nothing to compile - all classes are up to date.
  [INFO]
  [INFO] --- resources:3.3.1:testResources (default-testResources) @ distributed-systems-hw1 ---
  [INFO] Copying 1 resource from src/test/resources to target/test-classes
  [INFO]
  [INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ distributed-systems-hw1 ---
  [INFO] Nothing to compile - all classes are up to date.
  [INFO]
  [INFO] --- surefire:3.2.2:test (default-test) @ distributed-systems-hw1 ---
  [INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
  [INFO]
  [INFO] -------------------------------------------------------
  [INFO]  T E S T S
  [INFO] -------------------------------------------------------
  [INFO] Running HW2.ClientServerTests
  Client #1 sent: READ
  Client #1 sent: READ
  Server disconnected
  Client 1 received 1 messages:
  Hello from Server
  [INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.374 s -- in HW2.ClientServerTests
  [INFO] Running HW2.ClientTest
  Client #1 sent: READ
  Client #1 sent: WRITE Hello, Server! from Client #1
  Client #1 sent: READ
  Client #1 sent: WRITE Hello, Server! from Client #1
  Client #1 sent: WRITE Hello, Server! from Client #1
  Client #1 sent: WRITE Hello, Server! from Client #1
  Server disconnected
  Client 1 received 5 messages:
  Response 3
  Response 4
  Response 5
  Response 1
  Response 2
  [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in HW2.ClientTest
  [INFO]
  [INFO] Results:
  [INFO]
  [INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
```

References

1. https://site.mockito.org
2. https://www.w3schools.com/java/java_threads.asp
3. https://docs.docker.com/compose/
4. https://www.geeksforgeeks.org/java/socket-programming-in-java/
5. https://aws.amazon.com/builders-library/leader-election-in-distributed-systems/
6. https://learn.microsoft.com/en-us/azure/architecture/guide/design-principles/minimize-coordination
7. https://cloud.google.com/blog/topics/developers-practitioners/implementing-leader-election-google-cloud-storage
8. https://learn.microsoft.com/en-us/azure/architecture/patterns/leader-election
9. https://static.googleusercontent.com/media/research.google.com/en//archive/chubby-osdi06.pdf
10. https://www.hashicorp.com/en/blog/making-gossip-more-robust-with-lifeguard